CSE333 Data and Software Security - M1 Proposal

Project 8: API Security Testing Workbench (Student Grades Portal API)

Team: Team 10 (working alone for now, waiting for instructor approval)
Member: Mohammad Sawalhah - 2020904063
Date: 9 Nov 2026
Repository: https://github.com/SawalhahCode/YU-CSE333 (public, code only, the lab runs on localhost)

1. Team and roles

I am working alone until the instructor confirms. I do all the roles: project lead, developer, tester and documentation.

2. Problem statement

Students check their grades online and instructors enter and change them. Grades are private, so if a student can see other students' grades or change his own mark, this is a big problem for the university. Many APIs have this kind of bug because the developer checks the login but not the permission. In this project I build a small grades API, plant real OWASP API flaws in it on purpose, attack them, then fix them and prove the fix with tests.

3. Objectives

1. Build a small grades portal API with login (authentication), roles (authorization) and grades, using Python FastAPI and SQLite.
2. Plant 5 flaws from the OWASP API Security Top 10 (2023) on purpose (table in section 5).
3. Write Python attack scripts that prove each attack on the vulnerable version.
4. Fix each flaw with a small code change (one commit per fix) and show the diff.
5. Run the same attack scripts again on the fixed version and show that every attack is blocked.
6. Package everything as a lab: anyone can clone the repository and run the API and the attacks with Docker.

4. Scope

In scope: the API (backend) with simple business logic and a SQLite database, the 5 flaws below, attack scripts and functional test scripts, Docker setup.
Out of scope: UI (front-end), cloud deployment, real data, the other OWASP items.

These 5 flaws cover the 5 things Project 8 asks to harden: authorization (V1, V2), schema validation (V3), rate limits and secret management (V4), error handling (V5).

5. Planted flaws

V = planted vulnerability. SR = security requirement (the fix).

Flaw	Exploit (what the attacker does)	Fix (security requirement)
V1 API1 BOLA	Student A logs in normally, then changes the student ID in the request (`/students/102/grades` to `/students/103/grades`) and sees Student B's grades.	**SR1:** On every request the server checks that the logged-in user (from the token) owns the record.
V2 API5 BFLA	A student can GET his grade. He changes the request to PUT and puts a new `mark` in the body. The API accepts it because it only checks that he is logged in.	**SR2:** Every function checks the role. Only instructors can change grades (deny by default).
V3 API3 BOPLA	(a) While updating his phone number, a student adds `"gpa": 4.0` and the API saves it. (b) The class list returns the full student record (national ID, phone, password hash) when the instructor needs only name and grade.	**SR3:** Input accepts only the fields the user is allowed to change. Output returns only the fields the caller needs.
V4 API2 Broken Authentication	(a) No limit on login attempts, so passwords can be brute forced. (b) The JWT secret key is weak and written in the code, so the attacker signs a fake token with `role: instructor`.	**SR4:** Limit login attempts. Hash passwords (bcrypt). Strong secret key in an environment variable. Check token signature and expiry on every request.

V5 API8 Security Misconfiguration	Debug mode is left on, so a broken request returns the full stack trace (file paths, SQL, library versions). This helps the attacker plan the next step.	**SR5:** Debug off. Generic error message with an error ID. Details only in the server logs.

6. Initial requirements

Roles: Student and Instructor.

Functional requirements (FR):

- **FR1:** A user logs in with username and password and gets a token.
- **FR2:** A student views his own grades.
- **FR3:** A student views and updates his own profile (phone, password).
- **FR4:** An instructor views the class list with grades for his course.
- **FR5:** An instructor adds and changes grades.

Security requirements: SR1 to SR5 in the table above.

7. How I show before and after

One code base and one Docker setup on the main branch. The vulnerable version is tagged `vulnerable`. Each fix is one small commit with its own tag (`fix-1` to `fix-5`), so anyone can open the lab at any step and see the exact diff of each fix. The last step is tagged `hardened`.

In the demo, for each flaw: I run the attack script and the attack works, I show the diff of the fix, then I run the same script again and the attack is blocked.

For the secret (V4): the `vulnerable` version has a weak fake secret written in the code on purpose. In the fix the secret moves to an environment variable (`.env`, not in Git) and it is changed, so the old leaked secret does not work anymore.

8. Success criteria

- All 5 attacks work on `vulnerable` and are blocked on `hardened`, using the same attack scripts.
- The functional test scripts (FR1 to FR5) still pass after the fixes, so the fixes did not break the app.
- Every fix has a small diff that I can show and explain.
- Anyone can clone the repository and run the lab and all scripts with Docker Compose and one command.

9. Lab authorization and safety plan

- **Where:** in Docker on localhost (`127.0.0.1`) only, on my laptop or on any machine that clones the lab. Never open to the network or the internet.
- **Data:** only random sample data (fake students and grades generated by a script). No real names, passwords or personal data.
- **Targets and tools:** my Python attack scripts (run with pytest) target only the lab containers on localhost. No third party systems.
- **Secrets:** in a `.env` file that is not in Git. The only secret in Git is the fake weak one planted in the `vulnerable` version on purpose.
- **Repository:** public, with a warning in the README that the code is intentionally vulnerable and only for learning.
- **Authorization:** I own the laptop and every system I test. I ask the instructor to approve this plan.
- **Cleanup:** at the end of the course I stop and delete the containers and the test data.

10. Tools

Python, FastAPI, SQLite, Docker, Git/GitHub, pytest (runs the attack scripts and the functional test scripts).